Noetia — Development Process Document

Version 1.0 | May 2026

---

1. Overview

This document describes the end-to-end development process used to build Noetia over 12 months: from local development through code review, automated testing, CI/CD deployment, and incident response. The process is designed for a small team (4 people) that must ship frequently without sacrificing quality or production stability.

---

2. Repository Structure

Noetia uses a **monorepo** hosted on GitHub. All services — API, web, mobile, worker, image generation — live in a single repository under `services/`. This eliminates version drift between services and makes cross-service changes atomic.

```
noetia/
├── .github/
│   └── workflows/
│       ├── ci.yml          # Lint + test + build on every PR
│       └── cd.yml          # Deploy to production on push to main
├── services/
│   ├── api/                # NestJS backend
│   ├── web/                # Next.js web app
│   ├── mobile/             # React Native (Expo)
│   ├── image-gen/          # Python Flask microservice
│   └── worker/             # BullMQ job processor
├── infra/                  # Server setup scripts, Traefik config
├── docs/                   # Project documentation
├── docker-compose.yml      # Local development
├── docker-compose.prod.yml # Production resource limits overlay
├── docker-compose.server.yml # Production deployment
└── CLAUDE.md               # Developer guide (primary onboarding doc)
```

---

3. Git Workflow

Branching strategy

Noetia uses **trunk-based development** with a single long-lived `main` branch. Feature branches are short-lived and merged directly to `main` via pull request.

```
main  ──────────────────────────────────────────────── (production)
          ↗ feature/clubs-backend ↘
          ↗ fix/audio-streaming   ↘
          ↗ chore/migration-053   ↘
```

Why trunk-based: At a 4-person team size, long-lived branches create unnecessary merge conflicts and delay integration. Every merge to `main` triggers an auto-deploy, so the codebase is always in a shippable state.

Commit message format

```
<type>: <short description>

Types: feat | fix | chore | docs | ci | refactor
```

Examples:

```
feat: add phrase-anchored club discussions
fix: clamp heartbeat phraseDelta to zero to prevent negative stats
chore: add migration 053 — reading_stats table
docs: update CLAUDE.md with Whisper sync map procedure
ci: add concurrency group to CD to prevent parallel deploy races
refactor: extract TokenService from AuthService
```

Commit discipline

- **One logical change per commit.** Never batch unrelated changes.
- **Commit after every completed task.** Never let completed work sit uncommitted overnight.
- **Push immediately after every commit.** Prevents work loss and keeps the remote branch current.

Pull requests

All changes go through a pull request, even solo work. PRs serve as:

- A forcing function for self-review before merge
- The trigger for CI (lint + test + build)
- An audit trail for future developers

PR description template:

```
## What
[One paragraph describing what changed]

## Why
[Why this change is needed — ticket/sprint reference]

## Test plan
[How to verify this works — what to click, what to check]
```

---

4. Local Development

Prerequisites

Tool	Version	Notes
Docker	24+	All services run in Docker
Docker Compose	v2	`docker compose` (no hyphen)
Node.js	20 LTS	Only needed if running api/web outside Docker
Python	3.11+	Only needed if running image-gen outside Docker

Starting the development environment

```
# Clone the repo
git clone https://github.com/carloskfe/noetia.git
cd noetia

# Copy and fill in environment variables
cp .env.example .env
# Edit .env — see Environment Variables section in CLAUDE.md

# Start all services
docker compose up -d

# Verify all containers are running
docker compose ps
```

Services available after startup:

Service	URL
Web (Next.js)	http://localhost:3000
API (NestJS)	http://localhost:4000
MinIO Console	http://localhost:9001
Mailhog (email preview)	http://localhost:8025
Meilisearch	http://localhost:7700

Hot reload

Source directories are mounted as Docker volumes. Changes to TypeScript/TSX/Python files take effect without rebuilding containers. Changes to `package.json`, `Dockerfile`, or `tsconfig.json` require:

```
docker compose up -d --build <service>
```

Running migrations in development

```
docker compose exec api npm run migration:run
```

Seed data

```
# Seed book catalog from Gutenberg/Wikisource
docker compose exec api npx ts-node -r tsconfig-paths/register src/ingestion/seed-
ingestion.ts

# Seed audio stream URLs
docker compose exec api npx ts-node -r tsconfig-paths/register src/ingestion/seed-
```

```
audio-stream.ts

# Index books in Meilisearch
docker compose exec api npx ts-node -r tsconfig-paths/register src/search/seed-
search.ts
```

---

5. Testing Strategy

Philosophy

Every service has unit tests. Every service file has a corresponding test file. There are no exceptions. The test suite is the primary confidence mechanism that allows daily deployments to production without a manual QA phase.

Coverage threshold

All services enforce **≥ 80% coverage**. Coverage gates run as part of CI — a PR that drops coverage below 80% will fail the CI check and cannot merge.

Test file structure

Tests mirror the source structure exactly:

```
services/api/
├── src/
│   └── stats/
│       ├── stats.service.ts
│       └── stats.controller.ts
└── tests/
    └── unit/
        └── stats/
            ├── stats.service.spec.ts
            └── stats.controller.spec.ts
```

Test isolation rules

- **Mock all external dependencies:** No test touches a real database, Redis, MinIO, or external API.
- **No shared state:** Each test case is fully independent. `beforeEach` resets all mocks.
- **Use `jest.resetAllMocks()` in `beforeEach` — not `clearAllMocks()`.** Only `resetAllMocks` clears the `mockResolvedValueOnce` queue, preventing mock value bleedover between tests.
- **Dynamic dates in tests:** Never hardcode date strings. Use a helper like `utcDate(daysAgo)` that computes relative to the current test run date.

Running tests

```
# API — from repo root
docker compose exec api npm run test
docker compose exec api npm run test:cov

# Web
docker compose exec web npm run test
```

```
docker compose exec web npm run test:cov

# Image generation (Python)
docker compose exec image-gen pytest
docker compose exec image-gen pytest --cov=. --cov-report=term-missing
```

Types of tests written

Test type	Where	What's covered
Unit	`tests/unit/`	Service methods, controller routing, edge cases
Smoke (manual)	Browser	Golden path feature verification before release

No end-to-end tests (Cypress/Playwright) at current team size — coverage gates and manual smoke testing cover the gap.

---

6. CI/CD Pipeline

CI — Continuous Integration

Trigger: Every pull request to `main`, every push to `main`.

File: `.github/workflows/ci.yml`

Steps:

```
1. Checkout code
2. Set up Node.js 20
3. Install dependencies (api, web, worker, mobile)
4. Lint all TypeScript services (ESLint)
5. Type-check all TypeScript services (tsc --noEmit)
6. Run unit tests with coverage (api, web, worker)
7. Set up Python 3.11
8. Install Python dependencies (image-gen)
9. Run pytest with coverage (image-gen)
10. Build Docker images (api, web, image-gen, worker)
```

A PR cannot merge if any step fails.

Execution time: ~6–8 minutes on GitHub Actions free tier.

CD — Continuous Deployment

Trigger: Push to `main` (after CI passes).

File: `.github/workflows/cd.yml`

Steps:

```
1. SSH into production server (port 222, DEPLOY_SSH_KEY secret)
2. cd /opt/noetia && git pull origin main
3. docker compose --env-file .env.production \
     -f docker-compose.server.yml \
```

```
     up -d --build
4. docker compose exec -T api npm run migration:run:prod
5. docker image prune -f
```

Concurrency control: The CD workflow uses a concurrency group (`deploy-production`) with `cancel-in-progress: false` . This serializes deploys — if two pushes arrive simultaneously, the second waits for the first to complete rather than racing.

```
concurrency:
  group: deploy-production
  cancel-in-progress: false
```

Container cleanup: After deploy, the workflow runs a dynamic cleanup that removes stopped containers matching the `noetia-*` pattern:

```
docker ps -a --filter "name=noetia-" --filter "status=exited" \
  -q | xargs --no-run-if-empty docker rm
```

Deploy duration: ~4–6 minutes including image builds.

Deployment secrets

All secrets are stored in GitHub repo settings (Settings → Secrets → Actions):

Secret	Purpose
`DEPLOY_SSH_KEY`	Private SSH key for deploying to the production server
`SENTRY_AUTH_TOKEN`	Used during build to upload source maps to Sentry

Production environment variables (`DATABASE_URL` , `STRIPE_SECRET_KEY` , etc.) are stored in `.env.production` on the server at `/opt/noetia/.env.production` — never in GitHub.

---

7. Database Migration Process

Migrations are the most sensitive part of the deployment process. A bad migration can corrupt production data.

Creating a migration

```
# Generate a new migration from entity changes
docker compose exec api npm run migration:generate -- src/migrations/<timestamp>-
<Description>

# Or write a manual migration
# Create src/migrations/<timestamp>-<Description>.ts
# Implement up() and down() methods
```

Migration naming convention

```
<13-digit-timestamp>-<PascalCaseDescription>.ts

Examples:
1748000000000-CreateReadingStats.ts
1748100000000-AddPrivacySettings.ts
1748200000000-AddWeeklyGoals.ts
```

Migration review checklist

Before merging any migration:

- [ ] Migration is additive (adds columns/tables), not destructive
- [ ] If removing a column: the application code no longer references it and was deployed first
- [ ] Foreign keys have explicit `ON DELETE` behavior specified
- [ ] Any new index has a meaningful name (`idx_<table>_<column>`)
- [ ] `down()` method is implemented and tested locally
- [ ] Migration has been run locally: `docker compose exec api npm run migration:run`

Production migration execution

Migrations run automatically during CD (step 4 of the CD pipeline). If a migration fails, the deploy pipeline fails and the old containers continue running — the failure is visible in the GitHub Actions log.

To run migrations manually on the server:

```
ssh -p 222 root@84.247.140.175
cd /opt/noetia
docker compose --env-file .env.production -f docker-compose.server.yml \
  exec -T api npm run migration:run:prod
```

Migration history policy

The complete migration history (currently 55 migrations) is preserved in `CLAUDE.md` . Every migration is described in the table. This is the authoritative record — never delete or renumber migrations.

---

8. Definition of Done

A feature or task is **Done** when all of the following are true:

Code

- [ ] Feature works as specified in the acceptance criteria
- [ ] No TypeScript errors (`tsc --noEmit` passes)
- [ ] No ESLint errors (`eslint .` passes)

Tests

- [ ] Unit test file exists at the mirrored path under `tests/unit/`
- [ ] All tests pass (`npm run test` / `pytest`)
- [ ] Coverage for the modified service is ≥ 80%
- [ ] Tests cover: happy path, edge cases, and error scenarios

- [ ] No test depends on a real database or external network call

Review

- [ ] Pull request created with a description explaining what and why
- [ ] At least one team member has reviewed and approved
- [ ] All CI checks pass (lint, type-check, tests, build)

Deployment

- [ ] Merged to `main`
- [ ] CD pipeline ran successfully
- [ ] PM has verified the feature in production against acceptance criteria

---

9. Code Review Standards

What reviewers check

Correctness:

- Does the implementation match the acceptance criteria?
- Are edge cases handled (null, empty list, concurrent access)?
- Are errors returned with the correct HTTP status code?

Security:

- No user-supplied values concatenated into SQL (use TypeORM parameterized queries)
- No sensitive data logged (tokens, passwords)
- New endpoints protected by `JwtAuthGuard` unless explicitly public

Consistency:

- Follows existing patterns (module structure, DTO validation, repository access)
- New columns/tables follow naming convention (camelCase entity, snake_case DB via TypeORM)
- Tests follow the mirrored structure convention

Scope:

- No features added beyond what the ticket specifies
- No speculative abstractions or "we might need this later" code

What reviewers do NOT check

- Formatting — handled by ESLint/Prettier automatically
- Whether tests exist — blocked by CI coverage gate

Turnaround time

Reviews should be completed within 1 business day. If blocked on context, leave a comment and the author explains. PRs do not sit unreviewed for more than 24 hours.

---

10. Environment Management

Three environments

Environment	How to run	Database	Auth
Development	`docker compose up`	Local PostgreSQL	Dev JWTs, Mailhog
Preview	EAS build `--profile preview` (mobile)	Production (read-only test data)	Real auth
Production	Auto-deployed via CD	Production PostgreSQL	Real auth, real Stripe

There is no staging server — the single Contabo VPS runs production. Developers validate features locally and rely on the CI test suite for confidence before merge.

Environment variables

- **Development:** `.env` (gitignored, copied from `.env.example`)
- **Production:** `.env.production` on the server at `/opt/noetia/` (never committed, stored in password manager)
- **EAS (mobile):** Build-time env vars set as EAS secrets via `eas secret:create`

---

11. Incident Response

Severity levels

Level	Description	Example	Response time
P1	Production down	API unreachable, auth broken	Immediate
P2	Feature broken	Reader not loading, payments failing	< 2 hours
P3	Degraded	Slow search, image gen errors	< 24 hours

P1 response procedure

1. **Confirm** — check Grafana alert, `docker ps`, `docker compose logs -f api`
2. **Identify** — is it a new deployment? Check `git log` for recent merges
3. **Rollback if caused by code change:**

```
cd /opt/noetia
git revert HEAD --no-edit
git push
# Wait for CD to redeploy automatically (~5 min)
```

4. **Rollback if caused by migration:** Restore from daily backup (02:00 UTC PostgreSQL snapshot)
5. **Communicate** — post to Slack `#incidents` with status updates every 15 minutes
6. **Resolve** — once service is stable, write an incident report in Notion

Common failure modes and diagnosis

API 502 / Traefik can't reach container:

```
docker ps  # Is api container healthy?
docker inspect noetia-api-1 --format "{{.State.Health.Status}}"
```

```
docker compose -f docker-compose.server.yml logs --tail 50 api
```

Migration failed on deploy:

```
docker compose -f docker-compose.server.yml logs --tail 20 api
# Look for migration error — fix the migration, push, let CD redeploy
```

Disk usage alert (>80%):

```
df -h  # Check disk
docker image prune -a  # Clean unused images
docker system prune    # More aggressive (stops unused containers too)
```

Database connection exhausted:

```
# Check PostgreSQL connection count
docker compose -f docker-compose.server.yml exec db \
  psql -U noetia -c "SELECT count(*) FROM pg_stat_activity;"
# Restart api to release connections
docker compose -f docker-compose.server.yml restart api
```

Server access

```
# SSH (port 222 — changed from default 22)
ssh -p 222 root@84.247.140.175

# View live logs
docker compose -f docker-compose.server.yml logs -f api
docker compose -f docker-compose.server.yml logs -f web

# Access Grafana (SSH tunnel from local machine)
ssh -p 222 -L 3001:localhost:3001 root@84.247.140.175
# Then open http://localhost:3001

# NEVER paste multi-line content via SSH — use nano or base64 encoding
# See CLAUDE.md §Critical server operations for details
```

---

12. Documentation Practices

CLAUDE.md — Living developer guide

`CLAUDE.md` is the team's primary onboarding and operational document. It lives in the repo root and is updated whenever:

- A new service or module is added
- A new migration is applied
- An infrastructure procedure changes
- A hard-won lesson is discovered (e.g., SSH multi-line paste corruption)

It is the first document a new developer reads. It should always be current enough for a developer to set up a working environment from scratch in under 30 minutes.

Code comments policy

Comments are written only when the WHY is non-obvious:

- A hidden constraint or invariant
- A workaround for a specific external library bug
- A performance optimization that looks like a pessimization

Comments that explain WHAT the code does are not written — good naming makes that obvious.

PR descriptions

PR descriptions serve as the primary record of why a change was made. They should explain:

- The problem being solved
- Why this specific approach was chosen (if there are alternatives)
- Any risks or caveats

Document maintained by the DevOps Engineer and Backend Developer. Last updated: May 25, 2026.